

# Roteiro — Aula 1: back-end, Ruby e o primeiro Rails

---

**Deck:** `slides/build/aula-01-fundamentos-de-back-end.pptx` (69 slides, sendo 5 de prática) **Apostila da turma:** `01-fundamentos.md` · **Checkpoint:** `aula-01` **Antes de tudo:** `guia-do-instrutor.md` — conduzir a sala, não o conteúdo

---

## Antes de começar

---

### Na semana anterior

- [ ] Mandar `00-preparacao.md` no grupo e cobrar resposta de quem travou.
- [ ] Levantar quem usa Windows → WSL2 instalado **antes**.
- [ ] Cobrar o teste do Multipass (`multipass launch 24.04 --name teste`) — é da Aula 4, mas quem usa Windows precisa acertar a rede do WSL2 **antes**, e isso não se resolve em cima da hora.

### No dia, antes de a turma chegar

- [ ] Deck aberto, modo apresentador (as notas de cada slide estão preenchidas).
  - [ ] Dois terminais grandes: um para o `irb`, outro para o `rails`.
  - [ ] Fonte do terminal em pelo menos 18pt.
  - [ ] Insomnia aberto.
  - [ ] `curl -i https://api.seemxxiii.tech/api/v1/status` testado — é a sua demo do slide 27.
  - [ ] Gabarito em `~/capitacao-gabarito` na branch `aula-01`, pronto para projetar quem travar.
  - [ ] `gh auth status` funcionando no seu terminal — você vai demonstrar o `gh repo create`.
- 

## Cronograma

---

**As sete práticas são o esqueleto do dia.** Estão em negrito, e a regra é a de sempre: quando atrasar, corte **conteúdo**, nunca prática.

| Relógio      | Slides    | Bloco                                            | Min |
|--------------|-----------|--------------------------------------------------|-----|
| 00:00        | 1–3       | Abertura, objetivos e <b>o combinado de hoje</b> | 7   |
| 00:07        | 4–11      | Ferramentas e setup                              | 23  |
| 00:30        | 12–16     | O que é back-end                                 | 12  |
| 00:42        | 17–20     | Rede: servidor, IP, porta, DNS, URL              | 13  |
| 00:55        | 21–30     | HTTP, JSON e REST                                | 28  |
| 01:23        | <b>31</b> | <b>Prática 1 — HTTP na mão</b>                   | 10  |
| <b>01:33</b> | —         | <b>Intervalo</b>                                 | 10  |
| 01:43        | 32–49     | Ruby para quem sabe Python                       | 44  |
| 02:27        | <b>50</b> | <b>Prática 2 — Ruby no <code>irb</code></b>      | 10  |
| 02:37        | 51–61     | Rails, MVC, Zeitwerk, ambientes                  | 22  |
| 02:59        | <b>62</b> | <b>Práticas 3 e 4 — o projeto no ar</b>          | 25  |
| 03:24        | 63–64     | A primeira rota, e os dois diretórios            | 8   |
| 03:32        | <b>65</b> | <b>Prática 5 — a sua primeira rota</b>           | 20  |
| 03:52        | <b>67</b> | <b>Práticas 6 e 7 — teste e commit</b>           | 20  |
| 04:12        | 66, 68–69 | Git, recapitulação, fim                          | 5   |

**Isso dá 4h17, e é a aula mais cheia das quatro.** Não cabe em 3h sem você decidir antes o que sai. Corte **nesta ordem**, e decida na véspera:

| # | O que cortar                                                                                                                                | Ganho |
|---|---------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 1 | <b>O bloco 4–11 inteiro</b> (Ferramentas). O setup é <code>00-preparacao.md</code> , de casa — aqui você só roda a checagem final, em 3 min | –20   |
| 2 | Slides 42–44 (argumentos nomeados, <code>Struct</code> , exceções). Aparecem na Aula 3, no código real                                      | –10   |
| 3 | Slides 52 ( <code>O TERMINAL</code> ) e 66 ( <code>GIT</code> ), os dois marcados <code># CORTÁVEL</code>                                   | –7    |
| 4 | Slides 55 ( <code>DJANGO × RAILS</code> ) e 60 ( <code>TOUR DAS PASTAS</code> ) — ficam na apostila                                         | –6    |
| 5 | Slide 20 ( <code>DNS</code> ) — volta na Aula 4 de qualquer jeito                                                                           | –3    |
| 6 | Práticas 6 e 7: faça só o teste em aula, e mande o commit de casa                                                                           | –10   |

Cortando de 1 a 5, fecha em **3h31**. Cortando os seis, **3h21**.

O slide 46 ( `case` ) e a seta **não corte**: a seta aparece no model da Aula 2.

## Bloco a bloco

### 00:00 — Abertura e o combinado (1–3)

Você se apresenta e diz de onde veio a capacitação: **é a continuação da do Fiuza**. Lá foi o que o usuário vê; aqui é o outro lado.

Nos objetivos, a frase que prende: *"no fim do encontro 4, cada um de vocês vai ter uma URL `https://` própria, funcionando, que qualquer um do mundo consegue chamar."*

**E então o slide 3, que é o mais importante dos três primeiros minutos.** Não passe por ele rápido. A frase precisa sair da sua boca, olhando para a sala:

*"Vocês vão travar hoje. Todo mundo trava. Quando travar, levanta a mão — é para isso que eu estou aqui, e não para vocês fingirem que está tudo bem."*

Dita agora, ela dá permissão para a sala inteira. Sem ela, metade vai passar a aula escondendo que está perdida, e você só descobre na hora da prática. Detalhes em `guia-do-instrutor.md`.

### 00:07 — Ferramentas e setup (4–11)

Passe rápido pelos slides e **pare no 10**. Todo mundo roda os quatro comandos ao mesmo tempo:

```
curl https://mise.run | sh
mise use --global ruby@3.4.9
gem install rails -v 8.1.3
docker run --rm hello-world
```

**Peça para todos mostrarem a saída de `ruby -v` antes de seguir.**

**Ponto de não-retorno:** se passou de 00:35 e ainda tem gente instalando, **pare**. Pareie quem travou com quem terminou e siga. Instalação é problema de casa, não de aula.

### 00:30 — O que é back-end (12–16)

O slide 12 é a ponte com a capacitação anterior: mostre a coluna do front e diga "isso vocês já viram". O slide 13 tem o ponto que importa: **tudo que roda no navegador o usuário consegue ler e alterar** — por isso a validação que vale é a do back.

O restaurante (14) funciona bem. Volte nele quando falar de API.

### 00:42 — Rede (17–20)

Aqui muita gente descobre coisa que usava sem saber.

- **16:** servidor não é máquina especial, é programa esperando numa porta.

- **17**: a tabela de portas. Diga que 22, 80 e 443 voltam na Aula 4, quando forem abrir e fechar firewall na mão.
- **19**: desmonte a URL na tela. Pergunte: "por que `localhost:3000` tem dois pontos e `google.com` não?" Deixe alguém responder.

## 00:55 — HTTP e REST (21–30)

**Slide 22 é o coração do bloco.** Uma requisição HTTP é texto puro. Se der, faça ao vivo:

```
curl -v https://api.seemxxiii.tech/api/v1/status
```

O `-v` mostra as linhas com `>` (o que foi) e `<` (o que voltou). É a mesma coisa do slide, acontecendo de verdade.

No **24** (`Content-Type`), avise: "esquecer esse cabeçalho é o erro nº 1 de vocês na Aula 3. Vem 400 e a mensagem não ajuda em nada."

No **26** (status codes), a pergunta: "qual a diferença entre 401 e 403?" — 401 é "quem é você?", 403 é "sei quem você é, e você não pode". Diga que cai em entrevista.

No **fim do bloco**, marque a frase: **HTTP não tem memória**. Escreva no quadro se tiver. É o problema inteiro da Aula 3.

## 01:23 — Prática 1: HTTP na mão (31)

**A primeira vez que eles falam com um servidor sem navegador no meio.** Dez minutos, e circule.

O erro de condução aqui é deixar rodarem os cinco comandos em sequência sem ler nada. Pare depois do `-v` e peça, em voz alta, que alguém aponte onde termina a requisição e onde começa a resposta.

A pergunta de fechamento, que vale a prática inteira:

**"404 é erro de conexão?"**

Não: é uma resposta perfeitamente bem-sucedida dizendo que aquele recurso não existe. Quem entende isso hoje não confunde 404 com 500 na Aula 3.

Quem terminar rápido: o item (e), o POST na mão com `-X`, `-H` e `-d`. É o que o Insomnia faz por baixo.

## 01:43 — Ruby (32–49)

O bloco mais longo. **Não leia os slides — rode no `irb` ao vivo.**

```

3.class          # Integer
"oi".class       # String
:oi.class        # Symbol
nil.class        # NilClass

3.times { puts "oi" }

usuario = { nome: "Ana", role: :admin }
usuario[:nome]

[1, 2, 3].map { |n| n * n }
[1, 2, 3].select { |n| n.odd? }

```

Pontos onde vale parar:

- **35** — as três diferenças: `end`, `if` no fim da linha, retorno implícito.
- **37** — símbolo não existe em Python. A regra: conteúdo que o usuário lê é string; nome de coisa no código é símbolo.
- **40** — `map` / `select` são iguais aos de Python. A diferença é que aqui são o caminho normal.
- **41** — o `?` e o `!`. Diga que o Rails inteiro depende dessa convenção.
- **42-43** — argumentos nomeados e `Struct`. Fale a frase: "o `user:` sozinho não é erro de digitação", porque eles vão ver isso em todo service da Aula 3.
- **46** — `case` e a seta `->`. A seta aparece no model da Aula 2; sem este slide ela lê como símbolo mágico.
- **47** — as três pegadinhas. Rode no `irb`: `7 / 2`, depois `7.0 / 2`. E `puts 'Olá #{1+1}'` com aspas simples. São 30 segundos e economizam um bug cada.

## 02:27 — Prática 2: Ruby no `irb` (50)

Dez minutos no `irb`, e é a **única vez** na capacitação em que eles mexem em Ruby sem Rails no caminho. Não corte, mesmo atrasado.

Os oito passos estão na apostila (Prática 2). Os dois que fixam, e que valem cobrar em voz alta:

- **por que** `usuario["nome"]` deu `nil`? — a chave é o símbolo `:nome`, não a string
- **por que** `saudacao("Ana")` deu `ArgumentError`? — o método pede argumento nomeado

E o passo (b), que é o que mais surpreende quem vem de Python:

```
puts "entrou" if 0      # entra! Só nil e false são falsos em Ruby.
```

Quem terminar rápido: o item (h), escrever um módulo e incluir numa classe — é a Aula 2 chegando mais cedo.

## 02:37 — Rails (51-61)

- **52** é a ideia central: você não configura o óbvio, você segue o combinado.
- **55** (Django × Rails) — pergunte quem já usou Django ou Flask e ancore neles.

- **58** (Zeitwerk) — a frase: *"não é estilo, é mecanismo. Errou o caminho, a classe não existe."*
- **63** — mostre a rota e o controller lado a lado, e aponte a convenção ligando os dois.
- **64** (`DOIS DIRETÓRIOS`) — **não corte**. É onde fica claro que o repositório da capacitação é gabarito e que o projeto é deles. Sem isso, metade da turma vai editar o repo errado.

## 02:59 — Práticas 3 a 7: o projeto no ar (62, 65, 67)

A entrega da aula, em três blocos separados por slides curtos. **Ninguém sai sem os 200 na tela e sem o projeto no GitHub deles.**

**Prática 3 e 4 (slide 62, ~25 min)** — criar o projeto, subir banco e aplicação:

```
rails new automic_auth_api --api -d postgresql \
  --skip-action-mailbox --skip-action-text --skip-active-storage \
  --skip-jbuilder --skip-action-cable
cd automic_auth_api
docker compose up -d          # tem que ficar healthy
bin/rails db:prepare
bin/rails server               # e abrir /up
```

É onde mais gente trava, e são sempre os mesmos dois:

| Sintoma                                               | Saída                                                                                                                           |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <code>address already in use</code> na 5432           | <code>DB_PORT=5433 docker compose up -d</code> , e <code>export DB_PORT=5433</code> <b>no mesmo shell</b> do <code>rails</code> |
| <code>permission denied</code> no <code>docker</code> | <code>sudo usermod -aG docker \$USER</code> , e <b>sair e entrar de novo</b> — reabrir a aba não basta                          |

**Ponto de decisão:** se metade da turma não tiver o `/up` verde às 03:24, pare o conteúdo e resolva junto. Sem isso, as práticas 5 a 7 não acontecem.

**Prática 5 (slide 65, ~20 min)** — a rota. O erro clássico é `uninitialized constant Api::V1::StatusController`: o arquivo está no caminho errado. Volte ao slide 58 (Zeitwerk) e mostre a correspondência nome ↔ caminho na tela.

**Práticas 6 e 7 (slide 67, ~20 min)** — o teste e o commit. **Faça o passo de quebrar o teste de propósito** (`"ok"` → `"OK"`): são 30 segundos, e é o que muda a relação deles com teste.

Quem travar em qualquer uma: projete o arquivo do gabarito (`~/capacitacao-gabarito`, branch `aula-01`) e deixe a pessoa digitar. **Não mande copiar e colar** — o exercício inteiro são 20 linhas.

## 04:12 — Fecho (66, 68–69)

Recapitule em cinco frases (slide 68) e feche com o que vem: *"na próxima, a API ganha memória."*

## Perguntas que vão aparecer

| Pergunta                                                   | Resposta curta                                                                                                                                                                                      |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "Por que Ruby e não Python?"                               | Porque o <code>seem-backend</code> é Ruby, e porque Rails entrega um sistema inteiro sem você montar peça por peça. E a linguagem é o de menos: o que vocês vão aprender aqui vale em qualquer uma. |
| "Rails não morreu?"                                        | GitHub, Shopify e Basecamp rodam nele hoje. O que morreu foi o hype, não a ferramenta.                                                                                                              |
| "Preciso decorar os status codes?"                         | Não. Precisa saber a faixa: 2xx deu certo, 4xx você errou, 5xx eu errei. O resto se consulta.                                                                                                       |
| "Por que não usar o navegador para testar a API?"          | O navegador só sabe fazer GET. Nossas rotas são POST e DELETE.                                                                                                                                      |
| "Posso usar o Ruby que já está instalado no meu Linux?"    | Pode dar certo e pode dar errado. É por isso que existe o mise: a versão exata, por projeto.                                                                                                        |
| "O <code>--api</code> tira alguma coisa que vou precisar?" | Tira view, sessão de navegador e assets. Se um dia precisar, dá para trazer de volta — foi o que fizemos com os cookies.                                                                            |

## Dever de casa

1. Terminar a prática, se não deu tempo.
2. Ler `ruby-para-pythonistas.md` inteiro.
3. Bônus da Prática 7: fazer a rota devolver `RUBY_VERSION` e `Rails.version`, com asserção no teste.